

WEEK 14 – Monitoring Concepts

TASK 1 : Monitoring Concepts (Theory Task):

1. Why monitoring is essential in DevOps?

Monitoring is important in DevOps because it ensures system stability, performance, and availability. In a CI/CD environment, applications are deployed frequently, so monitoring helps track system health in real-time.

It helps to:

- Detect issues early
- Reduce downtime
- Improve performance
- Respond to incidents quickly

Monitoring creates a feedback loop between production and development, helping teams improve reliability and user experience.

2. 5 key metrics you would monitor in a production system.

CPU Usage

Indicates how much processing power is being used. High CPU usage can slow down applications.

Memory Usage (RAM)

Helps detect memory leaks or insufficient memory allocation.

Disk Usage

Ensures storage does not become full, which may crash applications.

Network Traffic

Monitors incoming and outgoing data. Useful for detecting unusual traffic spikes or attacks.

Application Response Time (Latency)

Measures how quickly the system responds to user requests.

3. Compare 3 popular monitoring tools (Prometheus, Datadog, Nagios, etc.)

Prometheus

Pros:

Open-source and free

Powerful time-series database

Strong integration with Kubernetes

Flexible alerting with Alertmanager

Cons:

Requires manual setup and maintenance

Limited long-term storage without additional tools

Datadog

Pros:

Cloud-based (easy setup)

Beautiful dashboards

AI-based anomaly detection

Supports logs, metrics, traces

Cons:

Expensive

Pricing increases with scale

Nagios

Pros:

Mature and stable tool

Good for infrastructure monitoring

Strong alerting capabilities

Cons:

Outdated UI

Complex configuration

Limited visualization compared to modern tools

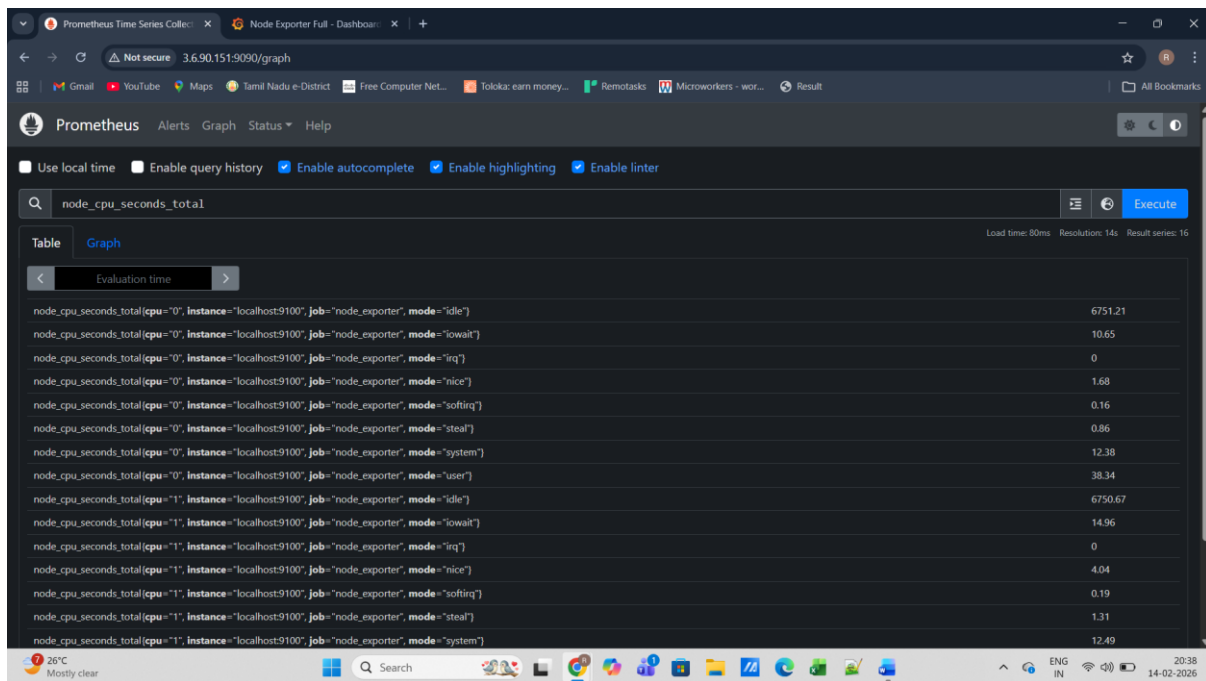
4. 3 best practices for effective monitoring.

- **Monitor What Matters** – Focus on business-critical metrics, not everything.
- **Set Proper Alert Thresholds** – Avoid alert fatigue by setting meaningful thresholds.
- **Use Centralized Dashboards** – Keep all important metrics in one place for quick visibility.

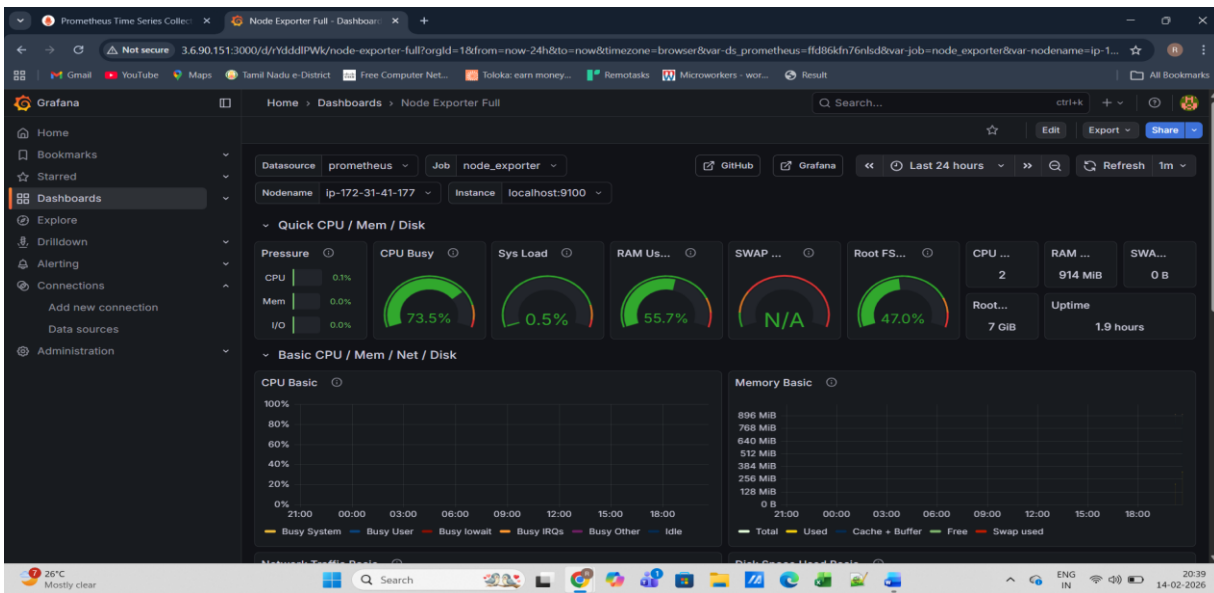
TASK 2 : Grafana Dashboard.

Installed Prometheus, Node exporter and Grafana.

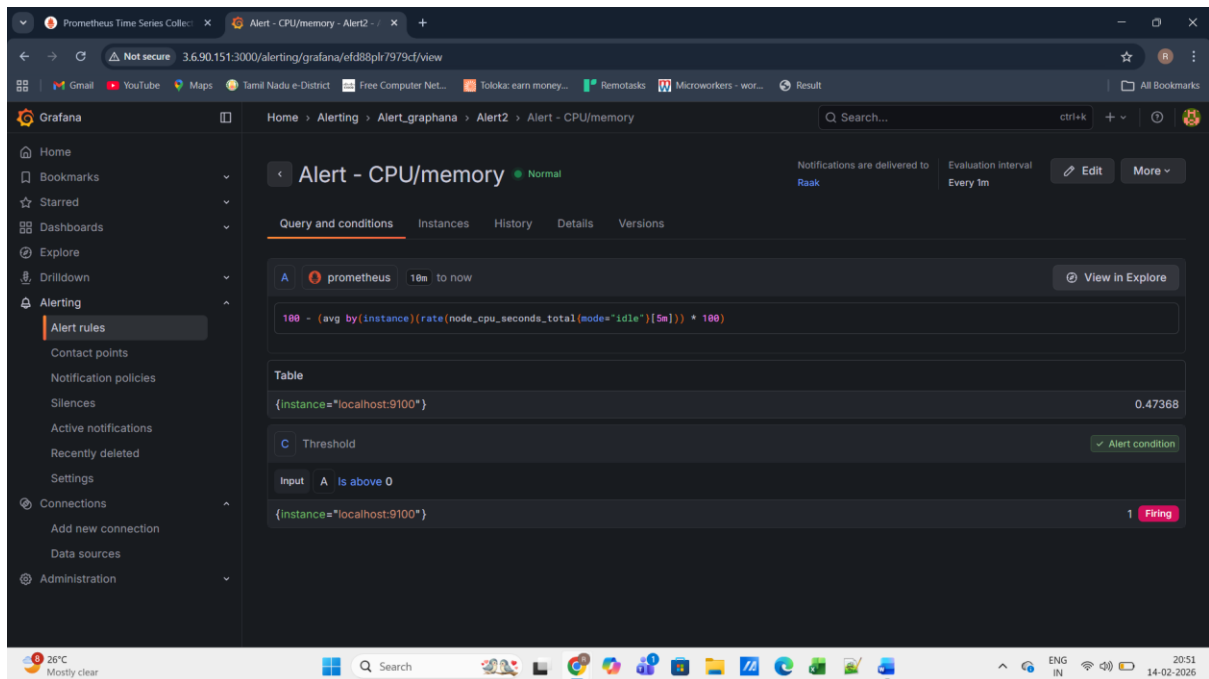
Prometheus:

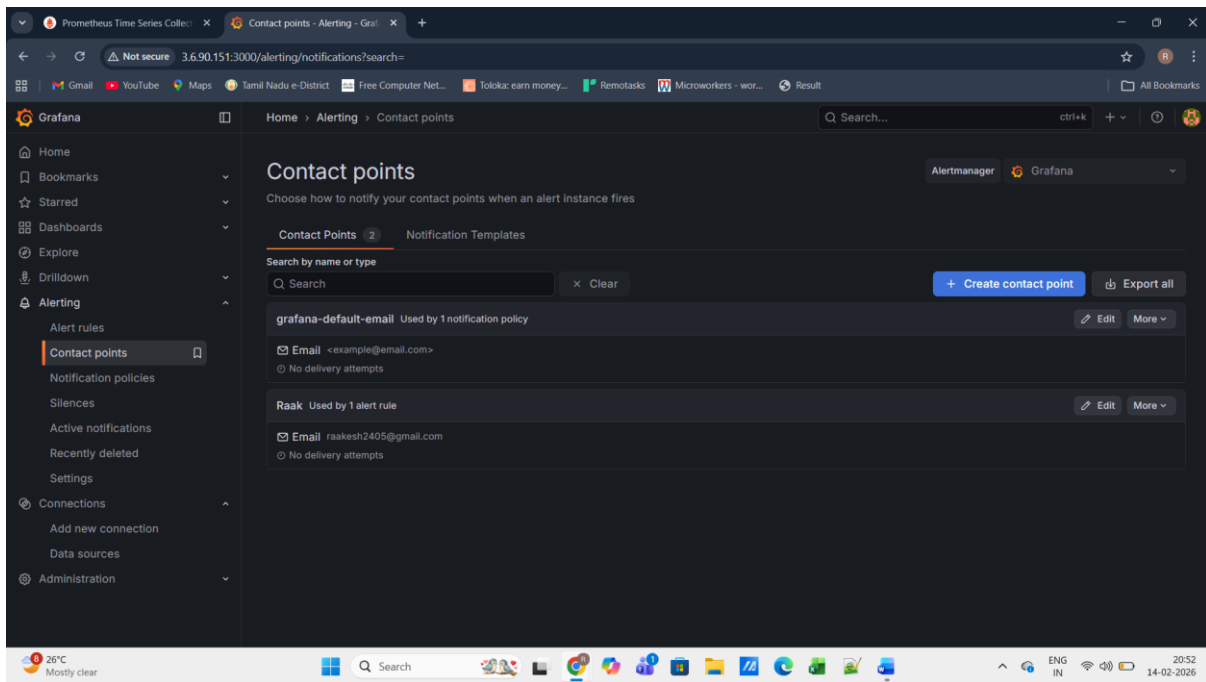


Grafana:



Task 3 –Alerting in Grafana





Task 4 :

1. Why Visualization is Important in Monitoring

In DevOps, monitoring systems generate a lot of data like CPU usage, memory usage, disk space, and network traffic. If we see these values only as numbers, it becomes difficult to understand the overall system health. Visualization helps convert these raw numbers into graphs, charts, and dashboards which are easier to understand.

Using tools like Grafana, we can clearly see patterns and trends over time. For example, if CPU usage suddenly increases, we can quickly identify it from a graph. This helps in faster decision-making and quicker problem resolution. Visualization also helps teams communicate better because everyone can understand dashboards easily without checking logs manually.

Overall, visualization improves system monitoring efficiency and reduces confusion during troubleshooting.

2. Three Best Practices While Creating Dashboards in Grafana

1. Keep the dashboard simple and clear

A dashboard should not be overloaded with too many panels. It should focus on important metrics only. Simple dashboards make it easier to understand system health quickly.

2. Use proper thresholds and color indicators

Using colors like green, yellow, and red helps identify system status immediately. For example, if CPU usage crosses 80%, it can be shown in red. This helps in identifying critical issues faster.

3. Organize related metrics properly

Metrics like CPU, memory, disk, and network should be grouped logically. This improves readability and makes analysis easier during troubleshooting.

3. How Alerts Reduce Downtime in Real DevOps Environments

Alerts play a very important role in preventing system failures. When a metric crosses a defined limit (for example, CPU usage above 80% for 5 minutes), Grafana sends an alert notification.

This allows the DevOps team to take immediate action before the system crashes. Early detection of issues reduces downtime and ensures high availability of applications. Alerts help maintain service reliability and improve overall system performance.

In real DevOps environments, proper monitoring and alerting ensure smooth operations and better user experience.